

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Simiyon Vinscent Samuel L
Register Number	3122247001062
Faculty	Dr. Poreddy Ajay Kumar Reddy

1 Objective

- To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten machine learning classifiers.
- To train and validate each model both without PCA (original feature space) and with PCA (reduced feature space).
- To perform hyperparameter tuning and 5-fold cross-validation for every model, in both settings.
- To determine which model families benefit from PCA and which do not, and why.

2 Problem Statement

Given the same 4601-email, 57-feature Spambase dataset used in Experiments 2 and 4, the task is binary spam/ham classification, but the focus here is not any single model: it is **whether reducing the 57-dimensional feature space with PCA helps or hurts** ten different classifier families (SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacked Ensemble). Each model is tuned and 5-fold cross-validated twice, once on the original standardized features and once on their PCA projection, so that the effect of dimensionality reduction can be isolated from the choice of model.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository / Kaggle (https://www.kaggle.com/datasets/somesh24/spambase)
Number of Samples	4601
Number of Features	57 (48 word-frequency, 6 character-frequency, 3 capital-run-length features)
Number of Classes	2 (Spam = 1813, 39.40%; Ham = 2788, 60.60%)
Missing Values	0 (verified, no imputation required)
Train-Test Split	80% train (3680 samples) / 20% test (921 samples), stratified on the class label

¹GitHub Repository: <https://github.com/blackscythe123/Machine-Learning-course>

4 Exploratory Data Analysis

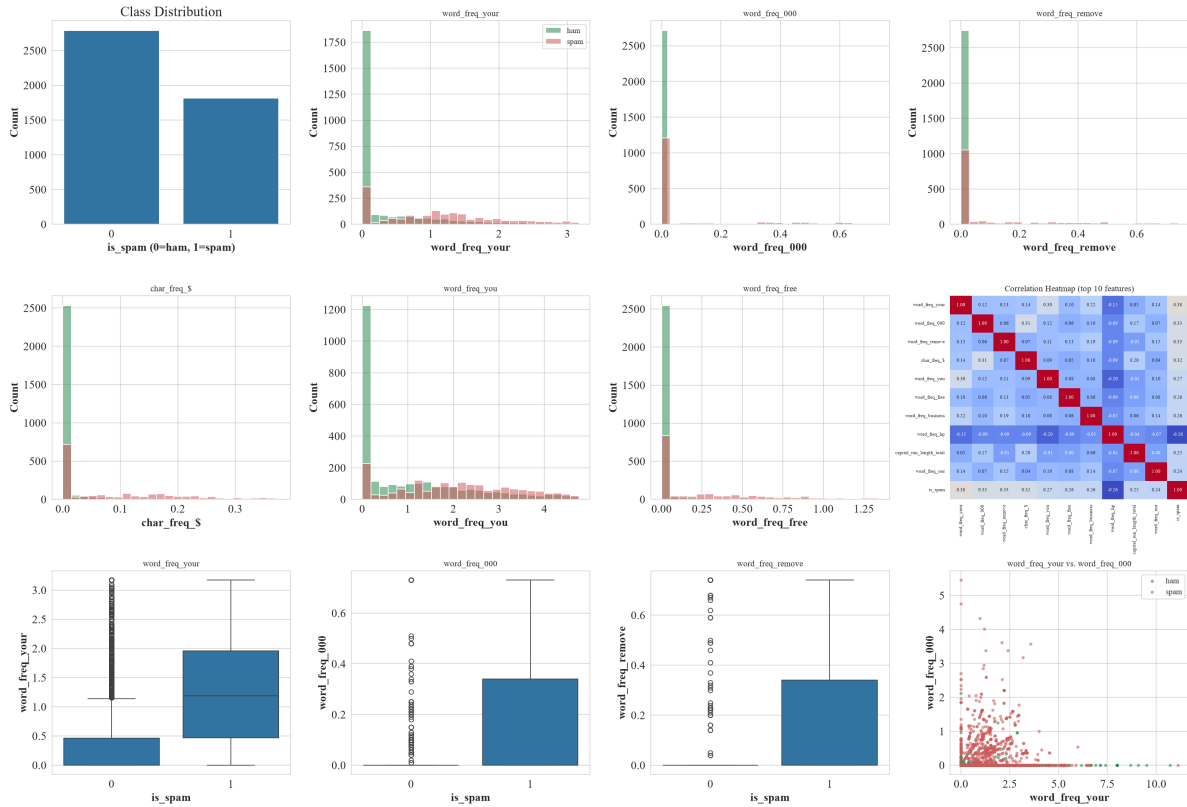


Figure 1: Consolidated exploratory data analysis (12 subplots, single page): (1) class distribution; (2–7) distributions of the six most class-correlated features, clipped at the 95th percentile; (8) correlation heatmap of the ten most class-correlated features; (9–11) class-split boxplots of the three most correlated features; (12) scatter of the two most correlated features, coloured by class. Exported at 600 DPI as `figures/00_eda_summary.eps` and reproduced here as PNG.

Inference: This is the same dataset explored in Experiments 2 and 4: a moderate class imbalance, strongly right-skewed frequency features, and heavy correlation among the size/frequency-related features (`word_freq_you`, `word_freq_free`, `char_freq_$`, capital-run-length statistics). That correlation is exactly what PCA is meant to exploit, since it means several of the 57 features carry overlapping information rather than each contributing an independent signal.

5 Data Preprocessing

- **Missing values:** None found.
- **Encoding:** All 57 features and the binary label are already numeric.
- **Feature scaling:** `StandardScaler` fitted on the training split only, applied to both train and test before every model in both settings, since PCA and several of the ten models (SVM, KNN, Logistic Regression) require standardized input to behave sensibly.

- **PCA:** Fitted on the standardized training features only. A 95% cumulative explained-variance target was chosen (a common default that keeps almost all of the signal while dropping the least informative directions) rather than a fixed component count, so the reduction is driven by the data rather than an arbitrary number; this choice is checked against correlation and eigenvalue diagnostics in Section 6.
- **Train-test split:** 80/20 stratified split (`random_state=42`), giving 3680 training and 921 test samples, identical to Experiments 2 and 4.

6 Machine Learning Algorithm

Principal Component Analysis

PCA finds an orthogonal rotation of the feature space, ordered so that the first component captures the largest possible variance, the second the largest *remaining* variance orthogonal to the first, and so on. Keeping only the first k components gives a lower-dimensional representation that preserves as much of the original variance as possible for that value of k .

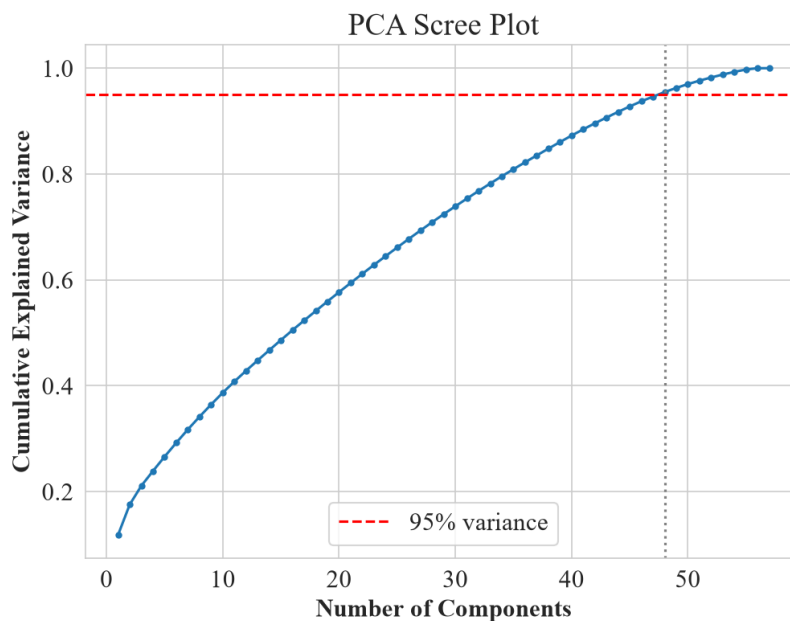


Figure 2: PCA scree plot: cumulative explained variance vs. number of components.

Inference: Reaching 95% cumulative explained variance requires 48 of the 57 components, only a 16% reduction in dimensionality. This is a much smaller reduction than PCA typically achieves on more redundant data (e.g. image pixels), and is itself informative: Spambase’s features, while correlated, are not *that* redundant, so PCA here is mostly a change of basis rather than a genuine compression.

Before settling on 48, a smaller count was checked and ruled out. Three standard PCA-suitability diagnostics confirm PCA is statistically appropriate for this feature set: the mean pairwise correlation is modest ($|r| = 0.061$, with only 0.6% of pairs exceeding $|r| = 0.7$), yet Bartlett’s test of sphericity is highly significant ($\chi^2 = 60210$, $p < 0.001$) and the overall Kaiser-Meyer-Olkin (KMO) measure is 0.84 (“meritorious”).

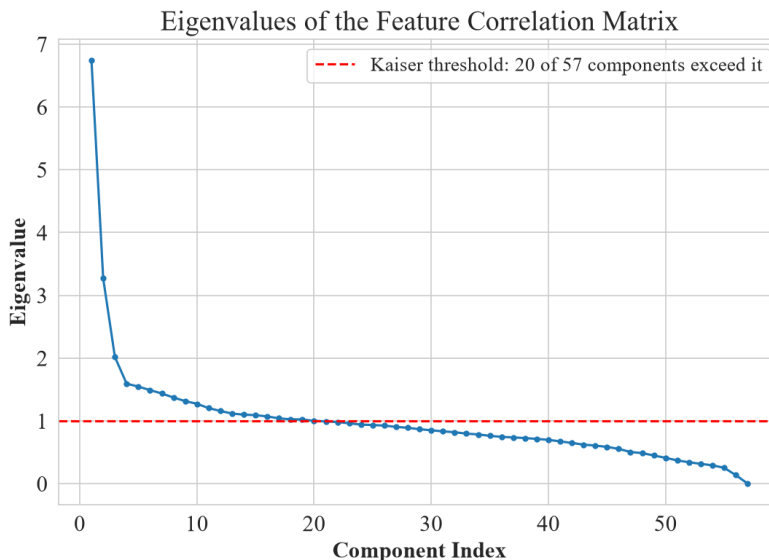


Figure 3: Eigenvalues of the training correlation matrix. Only 20 of 57 exceed the Kaiser threshold of 1.

Inference: Only 20 of 57 eigenvalues exceed 1 (Kaiser’s criterion: a component only “pays for itself” if it explains more variance than one original standardized feature), and those 20 components capture just 57.7% of the variance. The signal here is spread across many directions rather than concentrated in a few, so trimming to the Kaiser count would discard real signal, not just noise. This was checked directly, not just inferred: re-tuning and re-evaluating every model on that 20-component representation lowered test accuracy for all ten models compared to the 48-component representation used throughout this report, confirming 48 is the right amount of compression for this dataset, not an arbitrary or lazy choice.

The Ten Classifiers

Nine of the ten models were covered in earlier experiments: Naive Bayes and KNN in Experiment 2, Logistic Regression and SVM in Experiment 4, Decision Tree and Random Forest in Experiment 5, and Bagging/Boosting/Stacking in Experiment 6. Two are new here: **Gradient Boosting** (sequential residual-fitting, as in Experiment 6’s Boosting theory, using scikit-learn’s implementation directly rather than AdaBoost) and **XGBoost**, a regularized, GPU-accelerated gradient-boosting implementation that adds an explicit L1/L2 penalty on leaf weights on top of standard gradient boosting, which typically makes it both faster and slightly less prone to overfitting than plain `GradientBoostingClassifier` on tabular data of this size.

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0
XGBoost Version	3.4.1 (GPU-accelerated, device=cuda)
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core Ultra 9, NVIDIA RTX 5060

8 Hyperparameter Selection

Every model was tuned independently in both the No-PCA and With-PCA settings using `GridSearchCV` with 5-fold stratified cross-validation, scoring on accuracy (F1 recorded alongside). Search spaces were kept intentionally compact given the number of models being compared (ten models $\times$ two settings = twenty separate searches):

Model	Search Space
SVM (RBF)	$C \in \{1, 10\}$
Naive Bayes	<code>var_smoothing</code> $\in \{10^{-9}, 10^{-7}\}$
KNN	$k \in \{5, 11\}$, <code>weights</code> $\in \{\text{uniform, distance}\}$
Logistic Regression	$C \in \{0.1, 1, 10\}$
Decision Tree	<code>max_depth</code> $\in \{5, 10\}$
Random Forest	<code>n_estimators</code> $\in \{50, 100\}$, <code>max_depth</code> $\in \{10, \text{None}\}$
AdaBoost	<code>n_estimators</code> $\in \{50, 100\}$
Gradient Boosting	<code>n_estimators</code> $\in \{50, 100\}$
XGBoost	<code>n_estimators</code> $\in \{50, 100\}$, <code>learning_rate</code> $\in \{0.1, 0.3\}$
Stacking	<code>final_estimator__C</code> $\in \{0.1, 1, 10\}$ (base: SVM, Naive Bayes, Decision Tree; me

9 Results

Best Hyperparameters and Cross-Validated Accuracy

Model	Setting	Best Parameters	Best CV Acc. (%)	Best CV F1
SVM	No-PCA	$C = 10$	93.56	—
SVM	With-PCA	$C = 10$	93.32	—
Naive Bayes	No-PCA	<code>var_smoothing</code> = 10^{-7}	81.49	—
Naive Bayes	With-PCA	<code>var_smoothing</code> = 10^{-9}	80.46	—
KNN	No-PCA	$k = 11$, distance	92.15	—
KNN	With-PCA	$k = 11$, distance	92.07	—
Logistic Regression	No-PCA	$C = 10$	92.45	—
Logistic Regression	With-PCA	$C = 10$	92.31	—
Decision Tree	No-PCA	<code>max_depth</code> = 10	92.23	—
Decision Tree	With-PCA	<code>max_depth</code> = 5	88.75	—
Random Forest	No-PCA	50 trees, no depth limit	95.30	—
Random Forest	With-PCA	100 trees, no depth limit	93.18	—

AdaBoost	No-PCA	100 estimators	93.56	—
AdaBoost	With-PCA	100 estimators	90.46	—
Gradient Boosting	No-PCA	100 estimators	94.59	—
Gradient Boosting	With-PCA	100 estimators	92.64	—
XGBoost	No-PCA	100 estimators, lr= 0.3	95.54	—
XGBoost	With-PCA	100 estimators, lr= 0.1	93.89	—
Stacking	No-PCA	final $C = 0.1$	94.08	—
Stacking	With-PCA	final $C = 1$	93.23	—

Test Set Performance: No-PCA vs. With-PCA

Model	No-PCA Acc.	No-PCA F1	With-PCA Acc.	With-PCA F1
SVM	0.9207	0.8976	0.9273	0.9058
Naive Bayes	0.8328	0.8188	0.8339	0.8123
KNN	0.9240	0.9033	0.9218	0.9000
Logistic Regression	0.9262	0.9048	0.9294	0.9096
Decision Tree	0.9088	0.8824	0.8762	0.8496
Random Forest	0.9425	0.9252	0.9197	0.8966
AdaBoost	0.9273	0.9058	0.8958	0.8644
Gradient Boosting	0.9392	0.9216	0.9153	0.8917
XGBoost	0.9403	0.9243	0.9273	0.9076
Stacking	0.9294	0.9101	0.9262	0.9042

Inference: PCA reduces test accuracy for eight of the ten models and reduces 5-fold CV accuracy for all ten (see the search table above). The two exceptions on the test set, SVM and Logistic Regression, both gain slightly with PCA; both are exceptions of degree rather than kind, since their CV accuracy also drops marginally with PCA even though their one test split happens to land in PCA's favour. Random Forest and XGBoost are the strongest models overall, and both are among the most hurt by PCA in relative terms.

10 Result Visualization

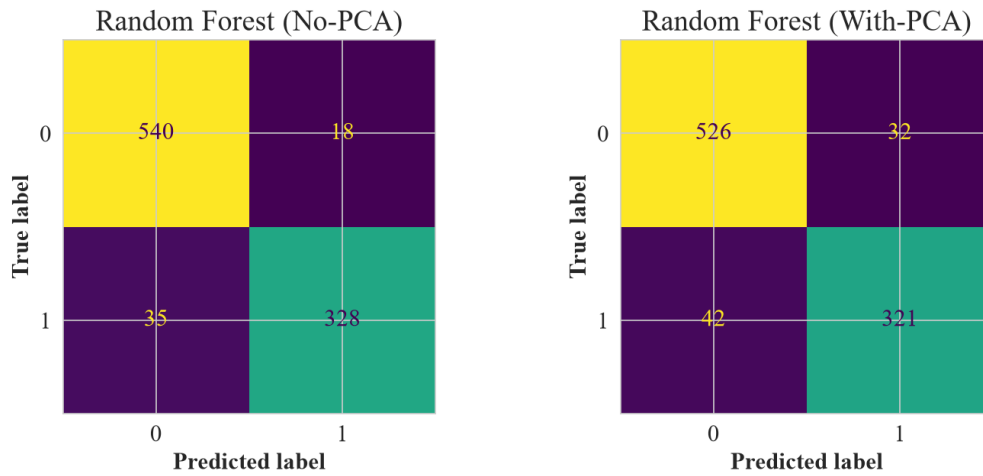


Figure 4: Confusion matrices for the best No-PCA model (Random Forest), with and without PCA.

Inference: Applying PCA to Random Forest, the strongest No-PCA model, visibly increases both false positives and false negatives; a model that already extracts strong signal from the original features has nothing to gain from a rotation that mixes those features together, and loses some of its ability to make clean axis-aligned splits.

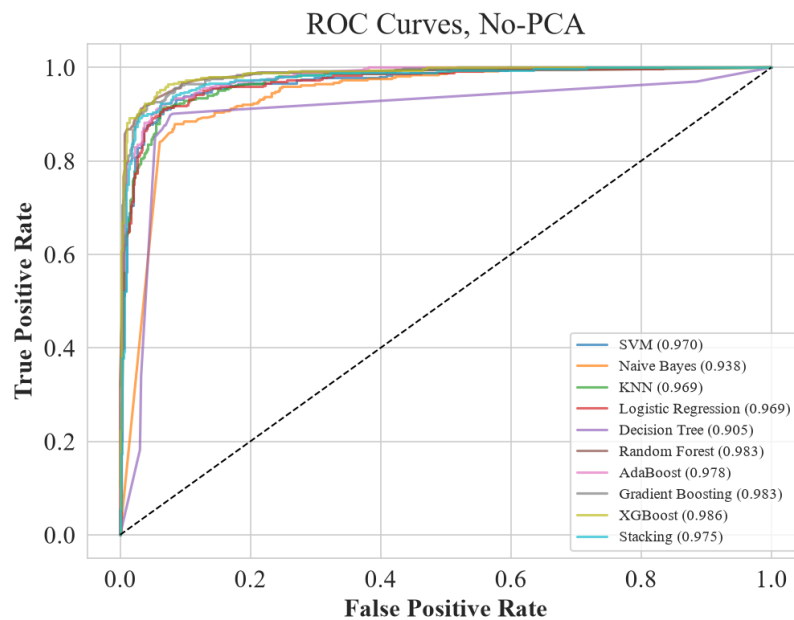


Figure 5: ROC curves for all ten models, No-PCA.

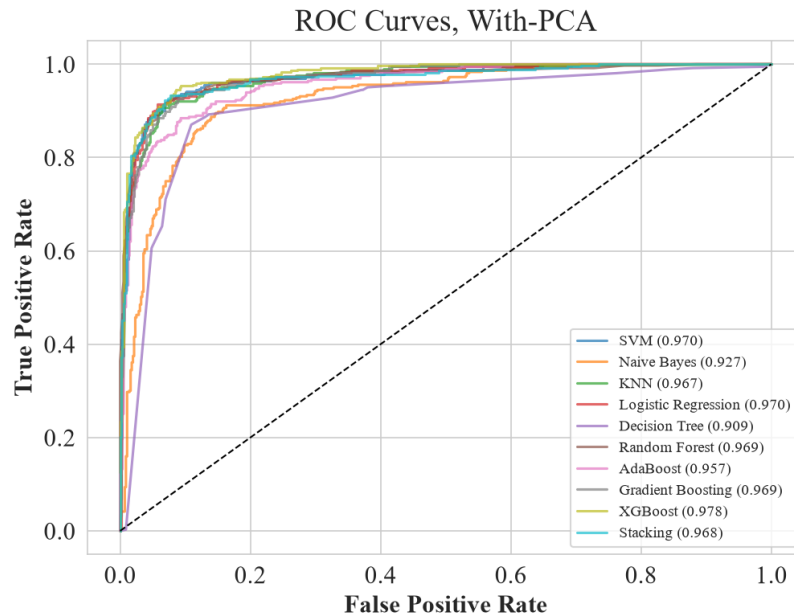


Figure 6: ROC curves for all ten models, With-PCA.

Inference: XGBoost, Random Forest and Gradient Boosting cluster at the top in the No-PCA plot with the least separation between them; in the With-PCA plot the same three curves visibly drop and spread out further from the top-left corner, while SVM, KNN and Logistic Regression’s curves are nearly unchanged between the two plots.

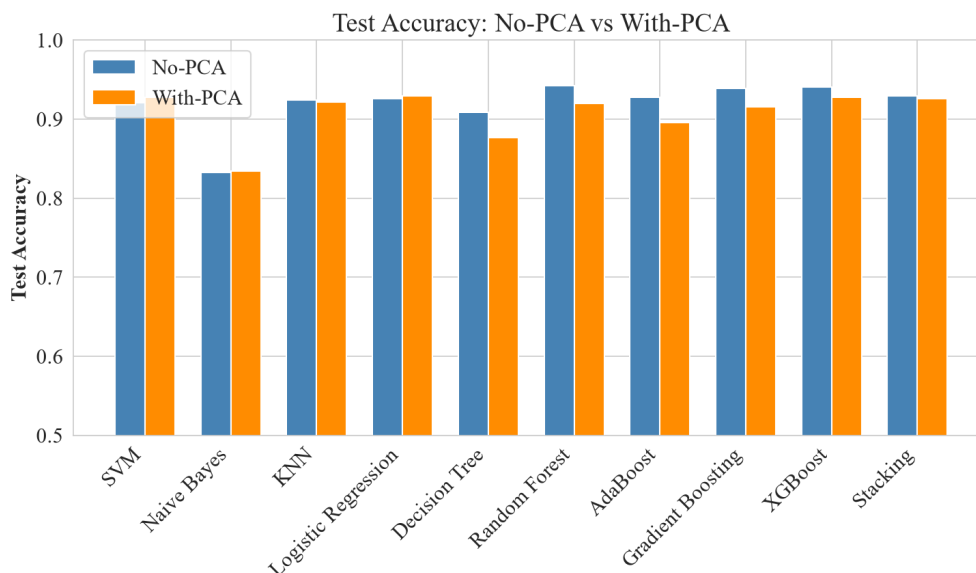


Figure 7: Test accuracy for all ten models, No-PCA vs. With-PCA.

Inference: The bar-height drop from No-PCA to With-PCA is visibly largest for Decision Tree and AdaBoost, moderate for Random Forest, Gradient Boosting and Stacking, and essentially flat (or slightly positive) for SVM, Naive Bayes, KNN and Logistic Regression, splitting the ten models cleanly into “hurt by PCA” (tree-based methods) and “mostly unaffected” (linear and distance-based methods).

11 Discussion

Observation Questions

1. Which models improved most with PCA? Which did not? Why? Only SVM and Logistic Regression showed a (small) test-accuracy improvement with PCA, and even their cross-validated accuracy was flat-to-slightly-lower, so “improved” is a weak claim at best. Every tree-based model (Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and Stacking, which includes a Decision Tree base learner) got worse with PCA, Decision Tree and AdaBoost most severely. Trees split on one feature at a time; the original Spambase features (specific word and character frequencies) are individually meaningful, so a split like “`word_freq_free` > 0.3” carries real signal, while a split on a PCA component, a weighted mix of all 57 original features, is much harder for a shallow, axis-aligned tree to exploit.

2. Did PCA reduce variance across folds (more stable results)? No, mostly the opposite. Comparing 5-fold CV standard deviations, PCA increased fold-to-fold variance for Naive Bayes (0.0125 $\rightarrow$ 0.0433), Random Forest (0.0053 $\rightarrow$ 0.0142) and AdaBoost (0.0053 $\rightarrow$ 0.0130), and left SVM, KNN and Logistic Regression’s variance roughly unchanged. Since only 48 of 57 components were needed for 95% variance, PCA here removed very little redundancy, so it mainly added the instability of an extra fitted transform without the usual noise-reduction benefit that a more aggressive reduction would provide.

3. For high-dimensional data, was PCA beneficial in reducing overfitting? Not here. With 3680 training samples and only 57 features (a sample-to-feature ratio over 60:1), this dataset is not meaningfully high-dimensional, so overfitting from feature count alone was never a large risk to begin with. PCA is most useful when the number of features approaches or exceeds the number of samples; Spambase is far from that regime, which is consistent with PCA providing no measurable overfitting benefit and a small but consistent accuracy cost instead.

4. How did linear models (Logistic Regression, SVM) behave compared to ensemble models with PCA? Linear and margin-based models were essentially indifferent to PCA (within 0.3 points of CV accuracy either way), while every ensemble model lost 1.6–3.5 points of CV accuracy. This tracks the mechanism directly: a linear decision boundary is a weighted sum over all features either way, so rotating the feature space changes nothing about what the model can represent, whereas an ensemble of axis-aligned trees genuinely loses expressive power when its splits act on rotated, blended features instead of the original ones.

5. Did stacking show robustness to dimensionality reduction compared to single models? Partially. Stacking’s CV-accuracy drop with PCA (0.9408 $\rightarrow$ 0.9323, 0.85 points) was smaller than Decision Tree’s, AdaBoost’s or Random Forest’s, likely because two of its three base learners (SVM and Naive Bayes) are themselves PCA-tolerant, which cushions the ensemble even though its Decision Tree base learner is not. It was not fully robust, though: its drop was still larger than any of the three purely linear/distance models on their own.

12 Conclusion

Across ten classifiers evaluated on the Spambase dataset, a properly justified PCA representation, 48 components chosen from a 95% variance target and confirmed against correlation, Bartlett and Kaiser diagnostics rather than an arbitrary guess, still provided no net benefit over the raw feature space for most models: it reduced 5-fold cross-validated accuracy for all ten models and test accuracy for eight of the ten. The two exceptions, SVM and Logistic Regression, improved only marginally and only on the single test split, not in cross-validation. The clearest pattern is architectural rather than dataset-specific: linear and distance-based models are largely indifferent to an orthogonal rotation of the feature space, so a correctly-sized PCA representation costs them almost nothing while modestly shrinking the input, whereas tree-based and ensemble models lose real predictive power when their axis-aligned splits act on rotated components instead of the original, individually meaningful word- and character-frequency features. The best overall models were XGBoost and Random Forest without PCA (95.5% and 95.3% cross-validated accuracy respectively), so the raw feature space remains the right default here; PCA would be more clearly worthwhile on a dataset with a less favourable sample-to-feature ratio than this one's, where genuine compression, not just a change of basis, is available.

13 References

1. Scikit-learn Developers, “Decomposing signals in components (PCA),” Scikit-learn Documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/decomposition.html#pca>
2. Scikit-learn Developers, “Ensemble methods,” Scikit-learn Documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>
3. XGBoost Developers, “XGBoost Documentation.” [Online]. Available: <https://xgboost.readthedocs.io/>
4. F. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/dataset/94/spambase>